

16 Cache and Persist in PySpark

Cache and Persist in PySpark

1. Introduction to Caching in Spark {#introduction}

What is Caching?

Caching (or persistence) in Spark is the process of storing intermediate results of RDD/DataFrame transformations in memory or disk for faster access in subsequent operations. This is crucial because Spark uses lazy evaluation - transformations are not executed until an action is triggered.

Why Cache?

- **Avoid Recomputation:** Without caching, Spark recomputes the entire lineage every time an action is called
- **Performance Optimization:** Accessing cached data is significantly faster than recomputing
- **Iterative Algorithms:** Essential for machine learning algorithms that iterate over the same dataset
- **Interactive Analysis:** Speeds up exploratory data analysis

Memory Management Context

Caching utilizes Spark's unified memory management system:

- **Storage Memory:** Used for caching RDDs/DataFrames
- **Execution Memory:** Used for shuffles, joins, sorts, aggregations
- **Dynamic Boundary:** Memory can be borrowed between storage and execution

2. Cache vs Persist {#cache-vs-persist}

cache()

```
# Simple caching - uses default storage level
df.cache()
# Equivalent to:
df.persist(StorageLevel.MEMORY_ONLY)
```

persist()

```
from pyspark import StorageLevel

# Explicit storage level specification
df.persist(StorageLevel.MEMORY_ONLY)
df.persist(StorageLevel.DISK_ONLY)
df.persist(StorageLevel.MEMORY_AND_DISK_SER)
```

Key Differences

Aspect	cache()	persist()
Storage Level	Fixed (MEMORY_ONLY)	Configurable
Flexibility	Limited	High
Use Case	Quick caching	Fine-tuned control

3. Storage Levels Deep Dive {#storage-levels}

3.1 MEMORY_ONLY

```
df.persist(StorageLevel.MEMORY_ONLY)
df.cache() # Default behavior
```

Characteristics:

- Stores data in JVM heap memory as deserialized objects
- Fastest access time
- No disk spillover - partitions are recomputed if they don't fit in memory
- **This is the default storage level for cache()**

Use Cases:

- Small to medium datasets that fit in memory
- Frequent access patterns
- High-performance requirements
- When you have abundant memory

Scenarios:

- Real-time analytics dashboards
- Interactive data exploration
- Machine learning feature engineering with small datasets
- Reference data lookups

Considerations:

- Risk of data loss if memory is full
- Can trigger recomputation if evicted
- Memory pressure on other operations

3.2 MEMORY_AND_DISK

```
df.persist(StorageLevel.MEMORY_AND_DISK)
```

Characteristics:

- Primary storage in memory, spillover to disk
- Deserialized objects in memory
- Automatic fallback mechanism

Use Cases:

- Most common caching scenario
- Large datasets that may not fit entirely in memory
- Balance between performance and reliability
- Production environments

Scenarios:

- ETL pipelines with multiple transformations
- Iterative algorithms (gradient descent, PageRank)
- Data preprocessing for machine learning
- Report generation with multiple aggregations

Considerations:

- Disk I/O overhead for spilled data
- Good balance of performance and safety
- Recommended for most use cases

3.3 MEMORY_ONLY_SER

```
df.persist(StorageLevel.MEMORY_ONLY_SER)
```

Characteristics:

- Serialized objects in memory using Pickle library (in Python)
- Compact representation
- CPU overhead for serialization/deserialization

- **Note:** In Python, all storage levels use Pickle serialization, so serialized vs non-serialized levels behave similarly

Use Cases:

- Memory-constrained environments
- Large datasets with limited memory
- When memory efficiency is more important than CPU

Scenarios:

- Large-scale batch processing
- Memory-limited cluster configurations
- Applications with high memory contention
- Complex objects with poor memory locality

Considerations:

- Higher CPU usage
- Slower access than MEMORY_ONLY
- Better memory utilization
- Risk of data loss if memory is full

3.4 MEMORY_AND_DISK_SER

```
df.persist(StorageLevel.MEMORY_AND_DISK_SER)
```

Characteristics:

- Serialized objects in memory with disk spillover using Pickle library (in Python)
- Most memory-efficient option with fault tolerance
- CPU overhead for serialization
- **Note:** In Python, serialization behavior is consistent across all levels

Use Cases:

- Large datasets in memory-constrained environments
- Production systems requiring fault tolerance
- When memory efficiency is critical

Scenarios:

- Large-scale data warehousing
- Complex ETL pipelines
- Batch processing with limited memory
- Long-running applications

Considerations:

- CPU overhead for serialization
- Excellent memory utilization
- Fault tolerant
- Slower than non-serialized options

3.5 DISK_ONLY

```
df.persist(StorageLevel.DISK_ONLY)
```

Characteristics:

- Stores data only on disk
- No memory usage
- Slower access than memory-based options

Use Cases:

- Very large datasets
- Memory-critical applications

- Checkpoint-like behavior
- When memory is needed for other operations

Scenarios:

- Massive batch processing jobs
- Data archiving and checkpointing
- Memory-limited environments
- Backup storage for critical computations

Considerations:

- Slowest access time
- No memory pressure
- I/O bound performance
- Good for infrequent access

3.6 OFF_HEAP (Experimental)

```
df.persist(StorageLevel.OFF_HEAP)
```

Characteristics:

- Stores data in off-heap memory (e.g., Tachyon/Alluxio)
- Reduces GC pressure
- Requires external storage system

Use Cases:

- Applications with GC issues
- Large-scale deployments
- When using external storage systems
- Experimental scenarios

Considerations:

- Requires additional setup
- Not widely adopted
- Experimental feature
- Complex configuration

3.7 Replication Levels

```
df.persist(StorageLevel.MEMORY_AND_DISK_2)
df.persist(StorageLevel.MEMORY_ONLY_2)
df.persist(StorageLevel.DISK_ONLY_2)
df.persist(StorageLevel.DISK_ONLY_3) # 3x replication available
```

Characteristics:

- Replicates data across 2 or 3 nodes
- Fault tolerance without recomputation
- Higher memory/disk usage

Available in Python:

- MEMORY_ONLY, MEMORY_ONLY_2
- MEMORY_AND_DISK, MEMORY_AND_DISK_2
- DISK_ONLY, DISK_ONLY_2, DISK_ONLY_3

Use Cases:

- Critical data that cannot be lost
- High availability requirements
- Expensive-to-compute datasets

- Production environments with fault tolerance needs

4. When to Use Caching {#when-to-use}

4.1 Multiple Actions on Same Dataset

```
# Without caching - recomputation occurs
df = spark.read.parquet("large_dataset.parquet")
df.filter(col("status") == "active").count() # Computation 1
df.filter(col("status") == "active").show()   # Computation 2

# With caching - computation happens once
df = spark.read.parquet("large_dataset.parquet")
filtered_df = df.filter(col("status") == "active").cache()
filtered_df.count() # Computation happens
filtered_df.show()  # Uses cached data
```

4.2 Iterative Algorithms

```
# Machine Learning - Gradient Descent
data = spark.read.parquet("training_data.parquet").cache()
for iteration in range(100):
    # Each iteration uses the same cached data
    gradient = compute_gradient(data, weights)
    weights = update_weights(weights, gradient)
```

4.3 Interactive Analysis

```
# Exploratory Data Analysis
df = spark.read.parquet("sales_data.parquet").cache()
df.describe().show() # Statistical summary
df.groupBy("region").count().show() # Regional analysis
df.filter(col("amount") > 1000).show() # High-value transactions
```

4.4 Branching Operations

```
# Multiple transformations from same base
base_df = spark.read.parquet("raw_data.parquet").cache()

# Branch 1: Aggregation
summary = base_df.groupBy("category").agg(sum("amount"))

# Branch 2: Filtering
filtered = base_df.filter(col("date") > "2023-01-01")

# Branch 3: Joining
enriched = base_df.join(reference_df, "id")
```

4.5 Checkpoint Alternative

```
# Expensive transformations
complex_df = (df
    .join(large_table1, "key1")
    .join(large_table2, "key2")
    .filter(complex_condition)
    .groupBy("group_key"))
```

```
.agg(complex_aggregation)
.cache()) # Cache to avoid recomputation
```

5. Performance Considerations {#performance}

5.1 Memory vs CPU Trade-offs

```
# Memory-optimized (faster access)
df.persist(StorageLevel.MEMORY_ONLY)

# CPU-optimized (less memory usage)
df.persist(StorageLevel.MEMORY_ONLY_SER)
```

5.2 Serialization Impact

Storage Level	Serialization	Memory Usage	CPU Usage	Access Speed
MEMORY_ONLY	No	High	Low	Fastest
MEMORY_ONLY_SER	Yes	Low	High	Medium
MEMORY_AND_DISK	No	Medium	Low	Fast
MEMORY_AND_DISK_SER	Yes	Low	High	Medium

5.3 Network and I/O Considerations

```
# For datasets accessed across multiple nodes
df.persist(StorageLevel.MEMORY_AND_DISK_2) # Replication reduces network traffic

# For disk-bound operations
df.persist(StorageLevel.MEMORY_AND_DISK) # Avoid pure disk storage
```

6. Best Practices {#best-practices}

6.1 Choose Appropriate Storage Level

```
# Small, frequently accessed data
reference_df.persist(StorageLevel.MEMORY_ONLY)

# Large datasets with iterative access
training_df.persist(StorageLevel.MEMORY_AND_DISK_SER)

# Extremely large datasets
big_df.persist(StorageLevel.MEMORY_AND_DISK)
```

6.2 Unpersist When Done

```
# Good practice: Clean up cached data
df.cache()
# ... use df multiple times ...
df.unpersist() # Non-blocking by default
df.unpersist(blocking=True) # Block until resources are freed
```

6.3 Automatic Cache Management

```
# Spark automatically manages cache with LRU eviction
# No need to manually manage unless you want explicit control
# Spark also auto-persists some shuffle operations internally
```

6.3 Cache at the Right Level

```
# Cache after expensive operations, before multiple uses
df = (spark.read.parquet("data.parquet")
      .filter(expensive_filter)
      .join(large_table, "key")
      .cache()) # Cache here, not at the beginning
```

6.4 Storage Level Selection Guidelines (from Spark Documentation)

```
# 1. If RDDs fit comfortably, use default MEMORY_ONLY
df.cache() # Uses MEMORY_ONLY - most CPU efficient

# 2. If not enough memory, try MEMORY_ONLY_SER (though in Python,
#     this behaves similarly due to Pickle serialization)
df.persist(StorageLevel.MEMORY_ONLY_SER)

# 3. Don't spill to disk unless:
#     - Functions that computed datasets are expensive
#     - They filter large amounts of data
#     - Otherwise, recomputing may be as fast as reading from disk

# 4. Use replicated storage levels for fast fault recovery
df.persist(StorageLevel.MEMORY_AND_DISK_2) # For web applications
```

6.5 Monitor Memory Usage

```
# Check cached data
spark.catalog.cacheTable("table_name")
spark.catalog.uncacheTable("table_name")

# SQL interface
spark.sql("CACHE TABLE my_table")
spark.sql("UNCACHE TABLE my_table")
```

6.6 Partition Awareness

```
# Cache after repartitioning for better performance
df = (spark.read.parquet("data.parquet")
      .repartition(200) # Optimize partition count
      .cache())
```

7. Common Pitfalls {#pitfalls}

7.1 Caching Too Early

```
# Bad: Caching before transformations
df = spark.read.parquet("data.parquet").cache()
result = df.filter(col("value") > 100).select("id", "value")

# Good: Cache after transformations
df = spark.read.parquet("data.parquet")
filtered_df = df.filter(col("value") > 100).cache()
result = filtered_df.select("id", "value")
```

7.2 Not Unpersisting

```
# Memory leak: Never unpersist
def process_data():
    df = spark.read.parquet("data.parquet").cache()
    return df.count()

# Better: Proper cleanup
def process_data():
    df = spark.read.parquet("data.parquet").cache()
    try:
        return df.count()
    finally:
        df.unpersist()
```

7.3 Wrong Storage Level

```
# Bad: MEMORY_ONLY for large datasets
large_df.persist(StorageLevel.MEMORY_ONLY) # May cause OOM

# Good: MEMORY_AND_DISK for large datasets
large_df.persist(StorageLevel.MEMORY_AND_DISK)
```

7.4 Caching Unused Data

```
# Bad: Caching data used only once
df = spark.read.parquet("data.parquet").cache()
df.count() # Used only once

# Good: No caching for single use
df = spark.read.parquet("data.parquet")
df.count()
```

8. Practical Examples {#examples}

8.1 Machine Learning Pipeline

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression

# Load and cache training data
training_data = (spark.read.parquet("training.parquet")
                 .cache()) # Will be used multiple times

# Feature engineering
assembler = VectorAssembler(inputCols=["feature1", "feature2"], outputCol="features")
feature_data = assembler.transform(training_data).cache()

# Model training (iterative)
lr = LogisticRegression(featuresCol="features", labelCol="label")
model = lr.fit(feature_data)

# Cleanup
training_data.unpersist()
feature_data.unpersist()
```

8.2 ETL Pipeline

```
# Extract
raw_data = spark.read.json("raw_data.json")

# Transform and cache intermediate results
cleaned_data = (raw_data
                .filter(col("status").isNotNull())
                .withColumn("processed_date", current_date())
                .cache())

# Multiple outputs from same intermediate data
summary_report = cleaned_data.groupBy("category").count()
detail_report = cleaned_data.filter(col("amount") > 1000)
audit_report = cleaned_data.select("id", "processed_date", "status")

# Load
summary_report.write.mode("overwrite").parquet("summary")
detail_report.write.mode("overwrite").parquet("details")
audit_report.write.mode("overwrite").parquet("audit")

# Cleanup
cleaned_data.unpersist()
```

8.3 Real-time Analytics

```
# Reference data (changes infrequently)
reference_data = (spark.read.parquet("reference.parquet")
                 .persist(StorageLevel.MEMORY_ONLY))

# Stream processing
def process_batch(batch_df, batch_id):
    # Join with cached reference data
    enriched = batch_df.join(reference_data, "key")

    # Process and write
    result = enriched.groupBy("category").count()
    result.write.mode("append").parquet(f"output/batch_{batch_id}")

# Streaming query
streaming_df = spark.readStream.format("kafka").load()
query = streaming_df.writeStream.foreachBatch(process_batch).start()
```

9. Monitoring and Debugging {#monitoring}

9.1 Spark UI - Storage Tab

Monitor cached RDDs and DataFrames:

- **Storage Level:** Current storage configuration
- **Cached Partitions:** Number of partitions in cache
- **Memory Usage:** Amount of memory used
- **Disk Usage:** Amount of disk space used

9.2 Programmatic Monitoring

```
# Check storage level
print(df.storageLevel)

# Check if cached
print(df.is_cached)
```

```
# Storage information
storage_info = spark.sparkContext.statusTracker().getExecutorInfos()
for executor in storage_info:
    print(f"Executor {executor.executorId}: Memory {executor.memoryUsed}/{executor.maxMemory}")
```

9.3 Memory Pressure Detection

```
# Monitor GC time and frequency
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")

# Adjust cache behavior based on memory pressure
if memory_pressure_detected():
    df.persist(StorageLevel.MEMORY_AND_DISK_SER)
else:
    df.persist(StorageLevel.MEMORY_ONLY)
```

9.4 Performance Metrics

```
import time

# Measure cache effectiveness
start_time = time.time()
df.cache()
df.count() # Trigger caching
cache_time = time.time() - start_time

start_time = time.time()
df.count() # Use cached data
access_time = time.time() - start_time

print(f"Cache creation time: {cache_time}")
print(f"Cached access time: {access_time}")
print(f"Speedup: {cache_time / access_time:.2f}x")
```

Key Corrections Based on Official Documentation

Important Notes for Python Users:

- Default cache() behavior:** `cache()` uses `MEMORY_ONLY`, not `MEMORY_AND_DISK`
- Serialization in Python:** All stored objects use Pickle serialization, so serialized vs non-serialized levels behave similarly
- Available storage levels in Python:**
 - `MEMORY_ONLY`, `MEMORY_ONLY_2`
 - `MEMORY_AND_DISK`, `MEMORY_AND_DISK_2`
 - `DISK_ONLY`, `DISK_ONLY_2`, `DISK_ONLY_3`
- Automatic persistence:** Spark automatically persists some intermediate data in shuffle operations
- LRU eviction:** Spark automatically manages cache using least-recently-used fashion
- unpersist() behavior:** Non-blocking by default; use `blocking=True` to wait for resource cleanup

Summary

Caching and persistence are fundamental optimization techniques in PySpark that can dramatically improve performance for iterative algorithms, interactive analysis, and complex ETL pipelines. The key is understanding your data access patterns, memory constraints, and performance requirements to choose the appropriate storage level.

Key Takeaways:

- Use caching when data will be accessed multiple times

2. Choose storage levels based on dataset size and memory availability
3. MEMORY_AND_DISK is the safest default choice
4. Always unpersist cached data when no longer needed
5. Monitor memory usage and adjust storage levels accordingly
6. Cache at the right point in your transformation chain
7. Consider replication for critical data in production environments

Remember that caching is not always beneficial - it consumes memory and has overhead. Use it judiciously based on your specific use case and performance requirements.

Serialization vs Deserialization in Spark Storage

Deserialized Storage (MEMORY_ONLY)

Object in Memory:

```
Row (
  id: 123 (Integer object)
  name: "John" (String object)
  age: 30 (Integer object)
)
```

How it's stored: As actual Java/Python objects in JVM heap memory

✓ Advantages:

- Fastest access time
- No CPU overhead for conversion
- Direct object method calls
- Most CPU-efficient

✗ Disadvantages:

- Higher memory usage
- Object overhead in JVM
- May not fit in memory

Serialized Storage (MEMORY_ONLY_SER)

Byte Array in Memory:

```
[0x4A, 0x6F, 0x68, 0x6E, 0x00, 0x7B, 0x00, 0x1E, ...]
Compact binary representation
```

How it's stored: As serialized byte arrays using Pickle (Python) or Java serialization

✓ Advantages:

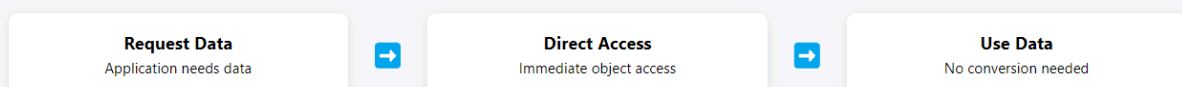
- Lower memory usage
- More data fits in memory
- Compact storage
- Better for large datasets

✗ Disadvantages:

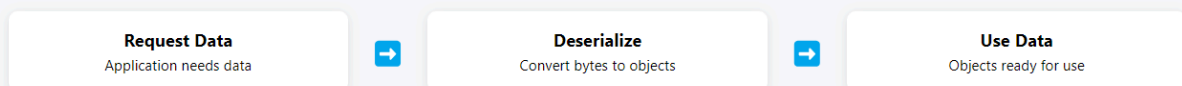
- CPU overhead for serialization
- Slower access time
- Must deserialize before use

Data Access Process

Deserialized Storage (MEMORY_ONLY):



Serialized Storage (MEMORY_ONLY_SER):



Aspect	Deserialized (MEMORY_ONLY)	Serialized (MEMORY_ONLY_SER)
Memory Usage	High	Low
Access Speed	Fastest	Medium
CPU Usage	Low	High
Storage Efficiency	Lower	Higher
Best For	Small datasets, frequent access	Large datasets, memory constraints

Special Note for Python

In PySpark, **all objects are serialized using Pickle** regardless of the storage level chosen. This means:

- `MEMORY_ONLY` and `MEMORY_ONLY_SER` behave very similarly
- The main difference is in how Spark internally manages the cached data
- Python objects are always converted to/from Pickle format
- The performance difference is less pronounced than in Scala/Java
- Python objects are always serialized using Pickle in PySpark, regardless of the serializer configuration, but JVM objects can use either Java serialization (default) or Kryo serialization depending on the `spark.serializer` setting. The `KryoSerializer` is generally faster and more compact than Java serialization for JVM objects, which is why it's often recommended for performance optimization, but it doesn't change how Python objects are handled.